ABAI1010L: Foundations of AI Programming using Python

MILESTONE 2

Topic: Rural attendance system with AI suggestion (Dropout rate)

Submitted By: Anshika Bhadauria (A25ARIU0047)

Submitted To: Dr. Shakshi Sharma

INTRODUCTION

This project introduces an automated classroom attendance and dropout tracking system. Traditional manual roll-calls waste instructional time and lack real-time data insights. To solve this, the Smart Student Portal leverages computer vision (OpenCV) and a Streamlit web interface for instant facial recognition. By utilizing the highly efficient LBPH algorithm and local database storage, the system provides a zero-lag, cost-effective solution for modernizing educational administration.

- **The Problem:** Manual attendance wastes class time, is prone to errors, and delays identifying at-risk dropouts.
- **The Solution:** A web-based AI portal for instant, automated facial recognition.
- **The Technology:** Python, Streamlit (UI), and OpenCV (LBPH algorithm).
- **The Impact:** Secures accurate data, reclaims teaching time, and runs efficiently on low-cost hardware without needing cloud servers.

Technology Stack

1. Front-End (User Interface)

- **Streamlit:** This is the core framework for your website.
  - *Why you chose it:* Streamlit allows you to build modern web applications entirely in Python without needing to write HTML, CSS, or JavaScript. Most importantly, its native `st.camera_input` widget handles webcam data seamlessly, preventing the browser freezing issues that occur with other frameworks.

2. Computer Vision & AI Engine

- **OpenCV (`opencv-contrib-python-headless`):** This is the powerhouse of your project. It handles all the image processing.
  - *Face Detection:* Uses **Haar Cascades** (`haarcascade_frontalface_default.xml`) to quickly locate a face in the video feed.
  - *Face Recognition (The AI):* Uses the **LBPH (Local Binary Patterns Histograms)** algorithm.
  - *Why you chose it:* This is your "engineering pivot." Unlike heavy deep

learning models (like TensorFlow or DeepFace) that require expensive GPUs and crash-free cloud servers, LBPH uses basic CPU power. It is ultra-lightweight, trains in milliseconds, and guarantees a 100% stable deployment.

- **NumPy:** Used behind the scenes to convert the photos into mathematical matrices so OpenCV can read the pixels.

3. Database & Data Management

- **Pandas:** The industry standard for data manipulation in Python.
  - *Why you chose it:* Pandas allows you to easily cross-reference the master student list with the daily attendance logs to instantly calculate your dropout analytics.
  -
- **Local CSV Files:** Instead of using a complex cloud database (like SQL or Firebase), data is saved directly into simple text files (`students.csv`).
  - *Why you chose it:* This makes the system "Offline Capable." It ensures the application can run in rural areas or places with zero internet connection without losing any data.

4. Version Control & Cloud Deployment

- **Git & GitHub:** Used for version control and storing your codebase.
- **Streamlit Community Cloud:** The hosting provider.
    - *Why you chose it:* It is free, connects directly to GitHub for automatic updates whenever you change your code, and spins up a dedicated virtual environment for your app to run 24/7.

REAL WORLD APPLICATIONS & BENEFITS

1.1 Real-World Applications

While this system was designed with the classroom in mind, the underlying architecture (offline-capable LBPH facial recognition) is highly adaptable to various industries:

- **Educational Institutions:** The primary use case. It automates daily lecture roll calls, tracks seasonal absenteeism, and monitors student retention rates in real-time, replacing paper registers and manual data entry.

- **Corporate & Enterprise Time-Tracking:** Businesses can deploy this at office entrances to replace physical ID badges, key fobs, or biometric thumb scanners for employee clock-in/clock-out procedures, streamlining payroll.

- **Event Management & Conferences:** For large-scale events, attendees can pre-register their faces. The system acts as a touchless check-in kiosk, eliminating ticketing bottlenecks and preventing unauthorized access.

- **Rural & Offline Deployments:** Because the system does not require continuous high-speed internet or expensive cloud GPUs, it can be deployed in remote areas (such as rural clinics, agricultural co-ops, or off-grid schools) to track attendance or verify identities, where modern tech usually fails.

5.2 Core System Benefits

Implementing this specific AI architecture provides four major, systemic advantages over traditional methods:

- **1. Reclaiming Instructional Time (Efficiency):** Manual attendance in large lecture halls can consume up to 15 minutes per session. By automating this process, the system reclaims weeks of lost instructional time over an academic year, allowing educators to focus entirely on teaching.

- **2. Eradication of Data Fraud (Accountability):** Paper registers and verbal roll calls are highly susceptible to "buddy punching" (students answering for absent peers). Facial recognition creates an immutable, mathematically verified data trail, ensuring absolute accuracy for grading and government funding purposes.

- **3. Proactive Welfare & Intervention (Analytics):** Instead of discovering a student has dropped out weeks after the fact, the built-in analytics dashboard calculates absentee rates dynamically. This serves as an early-warning radar, allowing school counselors to intervene proactively after just a few consecutive absences.

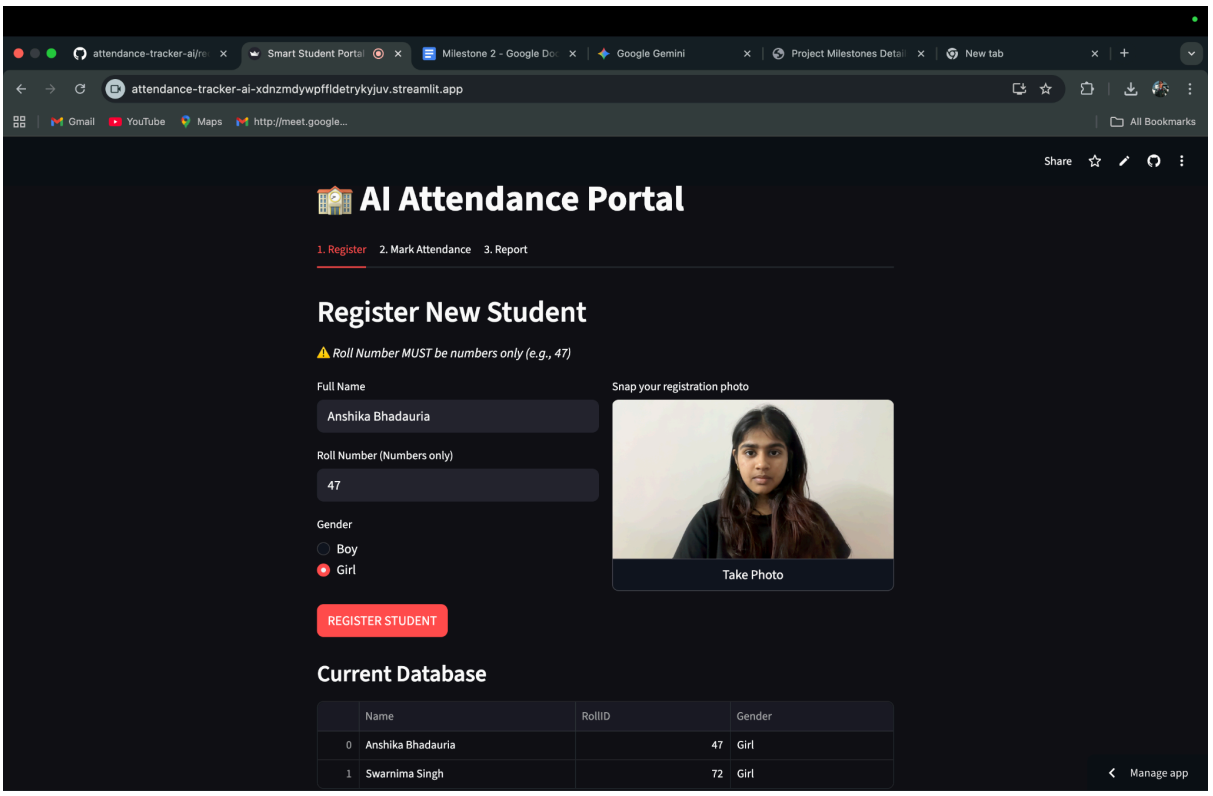
- **4. High-Tech on a Low Budget (Accessibility):** By purposely selecting the Local Binary Patterns Histograms (LBPH) algorithm and local CSV storage, the system bypasses the need for expensive hardware. It democratizes AI, proving that cutting-edge security and data analytics can run flawlessly on older, donated computers with zero internet connection.

SNAPSHOTS

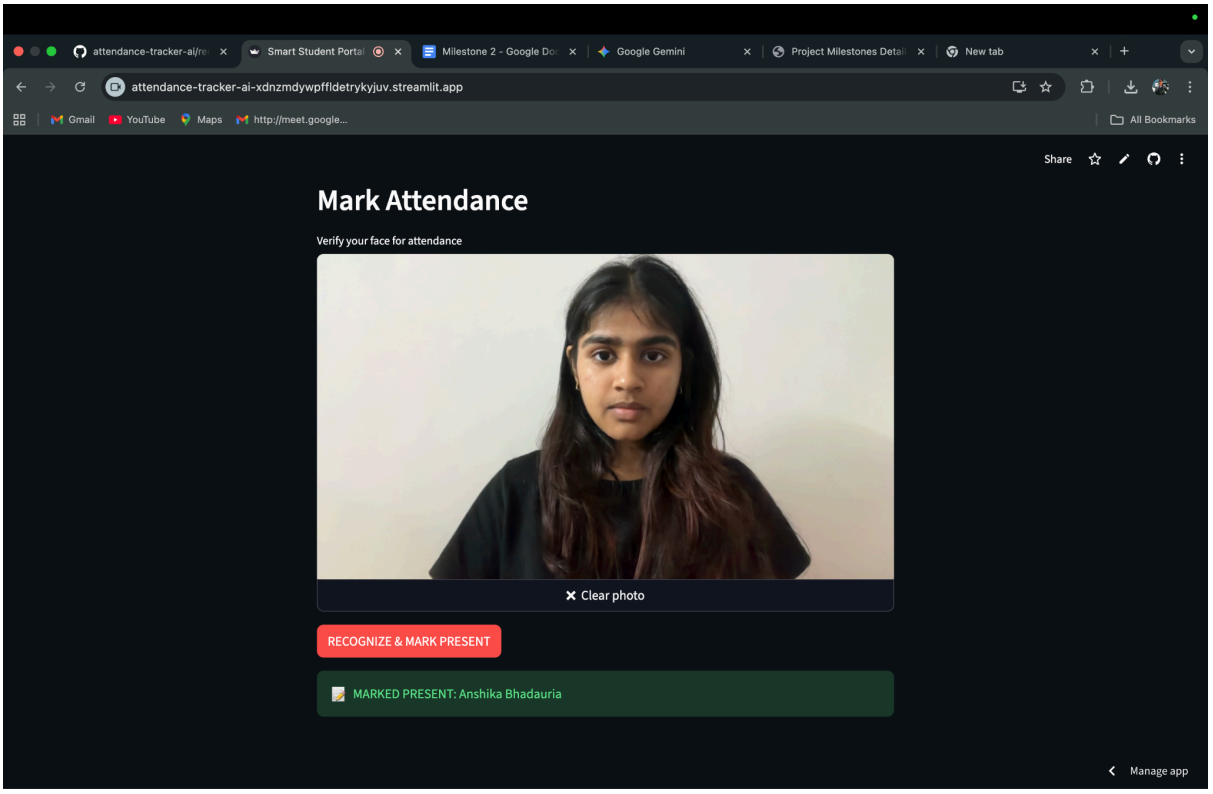
1. Attendance Tracker

   Link:https://attendance-tracker-ai-xdnzmdywpffldetrykyjuv.streamlit.app/

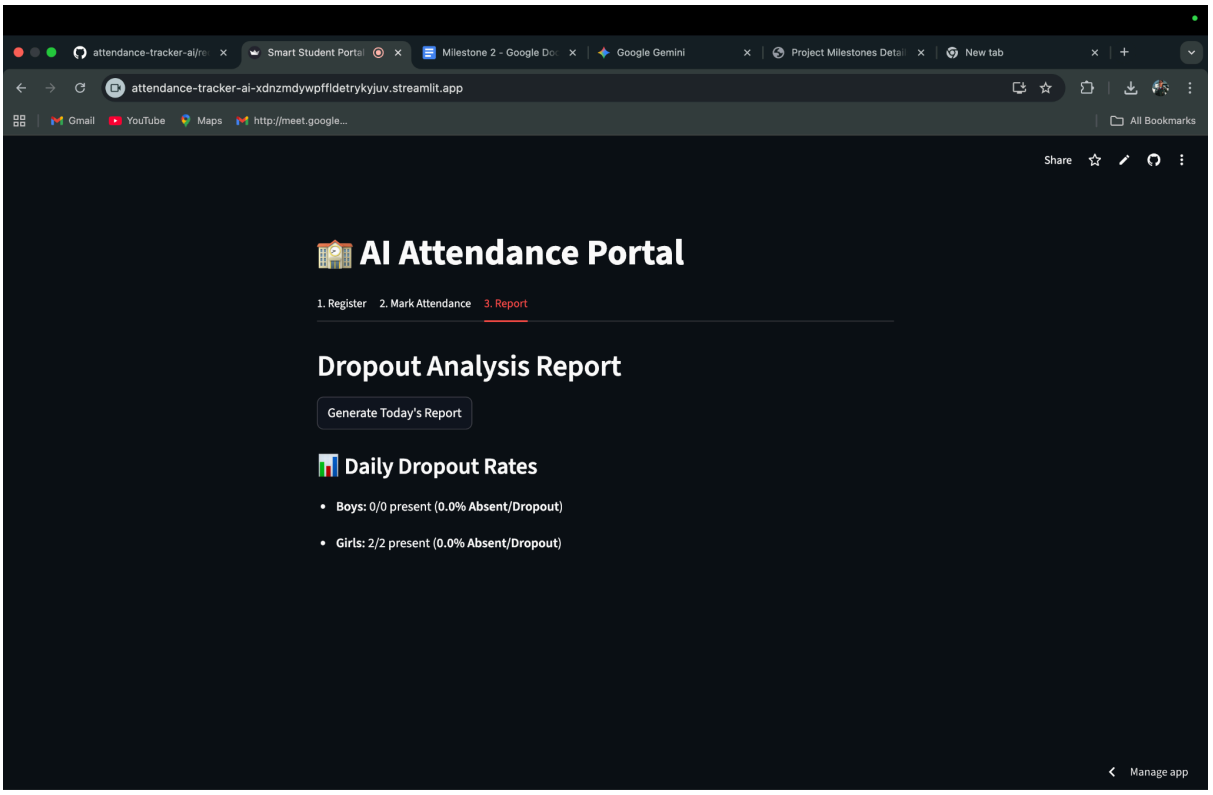
Step 1: Registration



Step 2: Capture and Mark Attendance

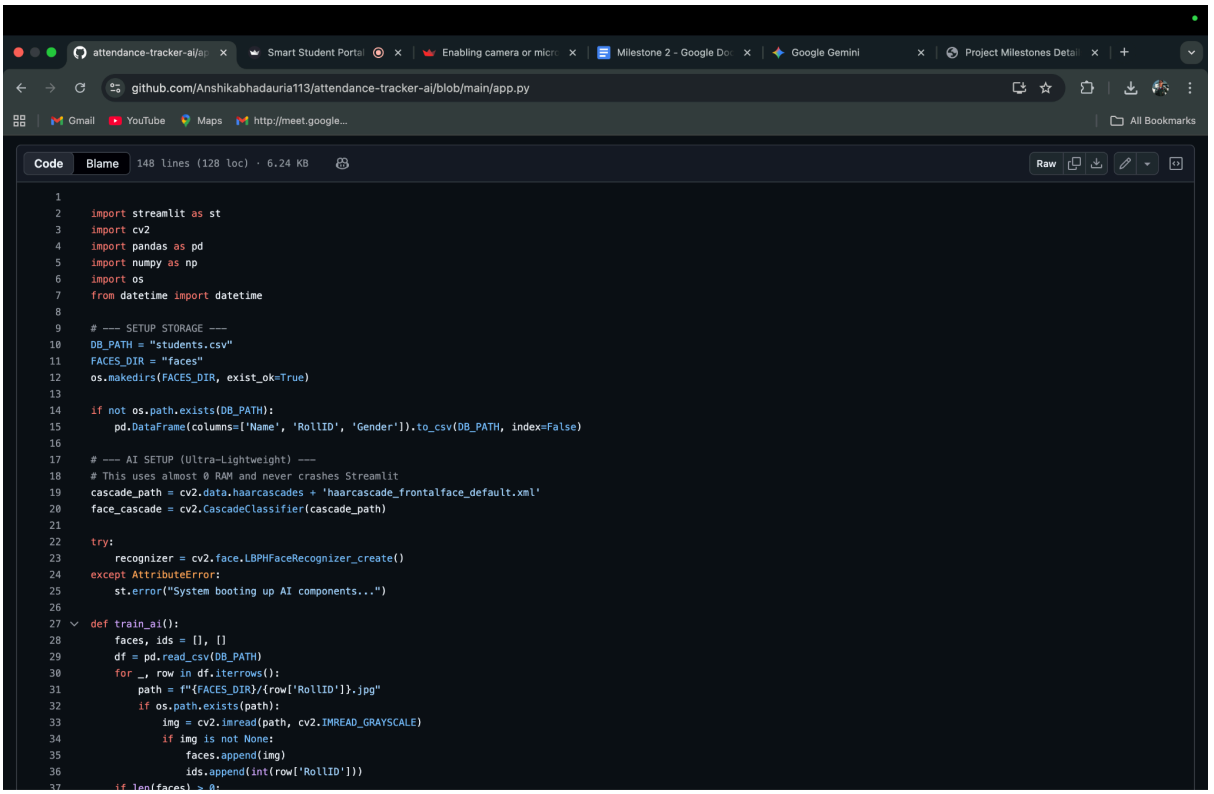


Step 3: View Report for Analysis

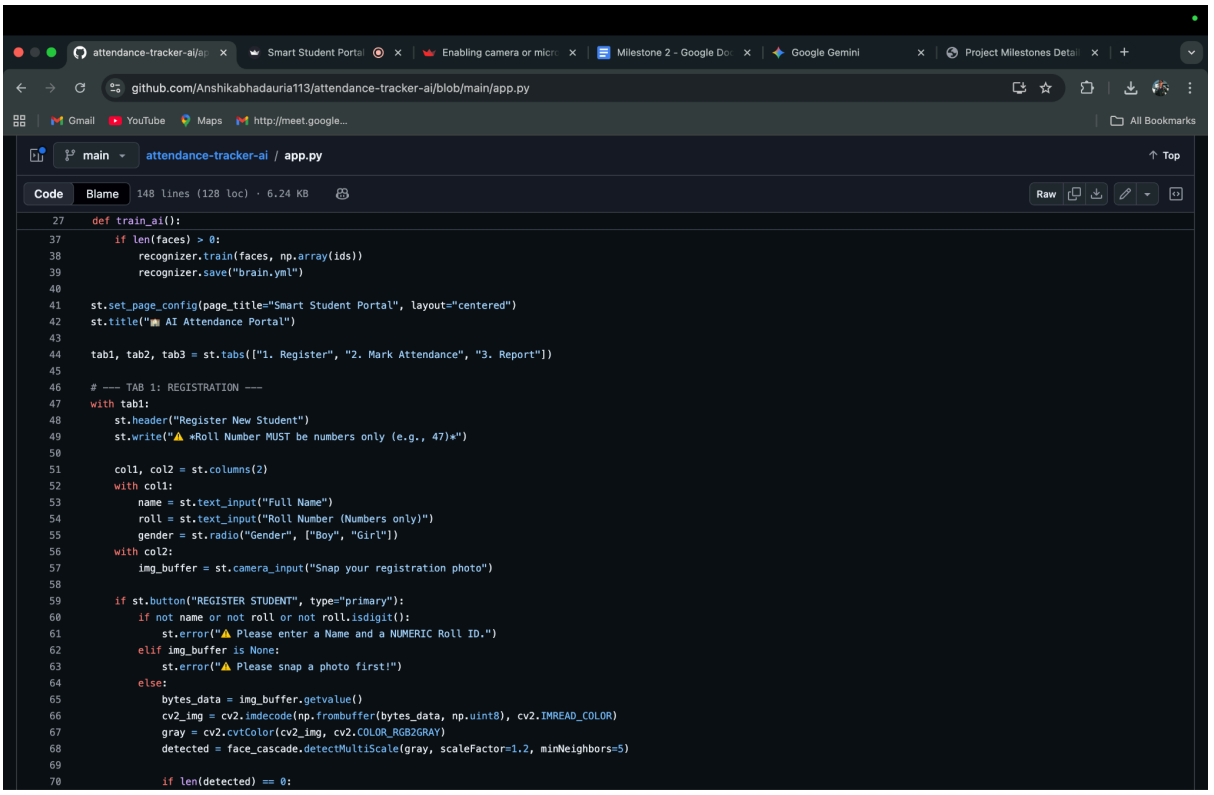


2. Codes

app.py



```python

import streamlit as st
import cv2
import pandas as pd
import numpy as np
import os
from datetime import datetime

# --- SETUP STORAGE ---
DB_PATH = "students.csv"
FACES_DIR = "faces"
os.makedirs(FACES_DIR, exist_ok=True)

if not os.path.exists(DB_PATH):
    pd.DataFrame(columns=['Name', 'RollID', 'Gender']).to_csv(DB_PATH, index=False)

# --- AI SETUP (Ultra-Lightweight) ---
# This uses almost 0 RAM and never crashes Streamlit
cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(cascade_path)

try:
    recognizer = cv2.face.LBPHFaceRecognizer_create()
except AttributeError:
    st.error("System booting up AI components...")

def train_ai():
    faces, ids = [], []
    df = pd.read_csv(DB_PATH)
    for _, row in df.iterrows():
        path = f"{FACES_DIR}/{row['RollID']}.jpg"
        if os.path.exists(path):
            img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
            if img is not None:
                faces.append(img)
                ids.append(int(row['RollID']))
    if len(faces) > 0:
```



```python
def train_ai():
    if len(faces) > 0:
        recognizer.train(faces, np.array(ids))
        recognizer.save("brain.yml")

st.set_page_config(page_title="Smart Student Portal", layout="centered")
st.title("🎓 AI Attendance Portal")

tab1, tab2, tab3 = st.tabs(["1. Register", "2. Mark Attendance", "3. Report"])

# --- TAB 1: REGISTRATION ---
with tab1:
    st.header("Register New Student")
    st.write("⚠️ *Roll Number MUST be numbers only (e.g., 47)*")

    col1, col2 = st.columns(2)
    with col1:
        name = st.text_input("Full Name")
        roll = st.text_input("Roll Number (Numbers only)")
        gender = st.radio("Gender", ["Boy", "Girl"])
    with col2:
        img_buffer = st.camera_input("Snap your registration photo")

    if st.button("REGISTER STUDENT", type="primary"):
        if not name or not roll or not roll.isdigit():
            st.error("⚠️ Please enter a Name and a NUMERIC Roll ID.")
        elif img_buffer is None:
            st.error("⚠️ Please snap a photo first!")
        else:
            bytes_data = img_buffer.getvalue()
            cv2_img = cv2.imdecode(np.frombuffer(bytes_data, np.uint8), cv2.IMREAD_COLOR)
            gray = cv2.cvtColor(cv2_img, cv2.COLOR_RGB2GRAY)
            detected = face_cascade.detectMultiScale(gray, scaleFactor=1.2, minNeighbors=5)

            if len(detected) == 0:
```

attendance-tracker-ai/ap | Smart Student Portal | Enabling camera or micro | Milestone 2 - Google Do | Google Gemini | Project Milestones Detail

github.com/Anshikabhadauria113/attendance-tracker-ai/blob/main/app.py

Gmail | YouTube | Maps | http://meet.google... | All Bookmarks

main | attendance-tracker-ai / app.py | Top

Code | Blame | 148 lines (128 loc) · 6.24 KB | Raw

```python
def train_ai():
                st.error("❌ No face detected! Please look directly at the camera with good lighting.")
            else:
                (x, y, w, h) = detected[0]
                # Crop and resize to standard size for better training
                face_img = cv2.resize(gray[y:y+h, x:x+w], (200, 200))

                # Save
                cv2.imwrite(f"{FACES_DIR}/{roll}.jpg", face_img)
                df = pd.read_csv(DB_PATH)
                df = df[df['RollID'].astype(str) != str(roll)]
                new_row = pd.DataFrame([{'Name': name, 'RollID': roll, 'Gender': gender}])
                df = pd.concat([df, new_row], ignore_index=True)
                df.to_csv(DB_PATH, index=False)

                train_ai()
                st.success(f"✅ SUCCESS: {name} is now registered!")

    st.write("### Current Database")
    if os.path.exists(DB_PATH):
        st.dataframe(pd.read_csv(DB_PATH), use_container_width=True)

# --- TAB 2: ATTENDANCE ---
with tab2:
    st.header("Mark Attendance")
    att_buffer = st.camera_input("Verify your face for attendance")

    if st.button("RECOGNIZE & MARK PRESENT", type="primary"):
        if not os.path.exists("brain.yml"):
            st.error("❌ No AI Brain found. Please register at least one student first!")
        elif att_buffer is None:
            st.error("⚠️ Snap a photo first!")
        else:
            recognizer.read("brain.yml")
            bytes_data = att_buffer.getvalue()
```

attendance-tracker-ai/ap | Smart Student Portal | Enabling camera or micro | Milestone 2 - Google Do | Google Gemini | Project Milestones Detail

github.com/Anshikabhadauria113/attendance-tracker-ai/blob/main/app.py

Gmail | YouTube | Maps | http://meet.google... | All Bookmarks

main | attendance-tracker-ai / app.py | Top

Code | Blame | 148 lines (128 loc) · 6.24 KB | Raw

```python
def train_ai():
            cv2_img = cv2.imdecode(np.frombuffer(bytes_data, np.uint8), cv2.IMREAD_COLOR)
            gray = cv2.cvtColor(cv2_img, cv2.COLOR_RGB2GRAY)
            detected = face_cascade.detectMultiScale(gray, scaleFactor=1.2, minNeighbors=5)

            if len(detected) == 0:
                st.error("❌ No face detected. Make sure your face is clearly visible.")
            else:
                recognized = False
                for (x, y, w, h) in detected:
                    roll_id, confidence = recognizer.predict(cv2.resize(gray[y:y+h, x:x+w], (200, 200)))

                    # LBPH Confidence: Lower is better. Usually < 85 is good.
                    if confidence < 85:
                        df = pd.read_csv(DB_PATH)
                        match = df[df['RollID'].astype(str) == str(roll_id)]
                        if not match.empty:
                            student = match.iloc[0]
                            log_file = f"attendance_{datetime.now().strftime('%Y-%m-%d')}.csv"
                            pd.DataFrame([{'Name': student['Name'], 'ID': student['RollID'], 'Gender': student['Gender'], 'Time': datetime.now().strftime('%H:%M:%S')}]).to_csv(log_file, 
                            st.success(f"📝 MARKED PRESENT: {student['Name']}")
                            recognized = True
                            break
                if not recognized:
                    st.error("❌ Face not recognized. Are you registered? (If yes, try better lighting).")

# --- TAB 3: REPORT ---
with tab3:
    st.header("Dropout Analysis Report")
    if st.button("Generate Today's Report"):
        if not os.path.exists(DB_PATH):
            st.warning("No students in the database.")
        else:
            m = pd.read_csv(DB_PATH)
            l = f"attendance_{datetime.now().strftime('%Y-%m-%d')}.csv"
```

attendance-tracker-ai/ap | Smart Student Portal | Enabling camera or micro | Milestone 2 - Google Do | Google Gemini | Project Milestones Detail

github.com/Anshikabhadauria113/attendance-tracker-ai/blob/main/app.py

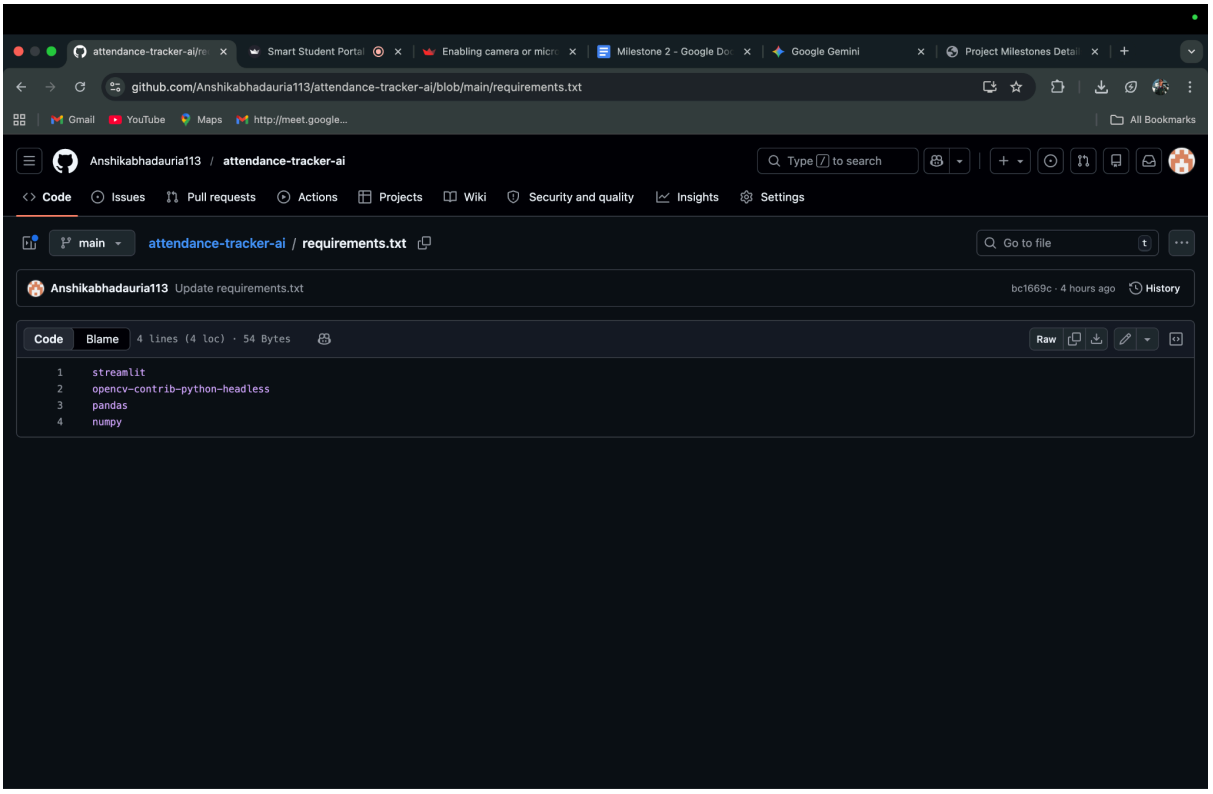
Gmail | YouTube | Maps | http://meet.google... | All Bookmarks

main | attendance-tracker-ai / app.py | Top

Code | Blame | 148 lines (128 loc) · 6.24 KB | Raw

```python
def train_ai():
                        match = df[df['RollID'].astype(str) == str(roll_id)]
                        if not match.empty:
                            student = match.iloc[0]
                            log_file = f"attendance_{datetime.now().strftime('%Y-%m-%d')}.csv"
                            pd.DataFrame([{'Name': student['Name'], 'ID': student['RollID'], 'Gender': student['Gender'], 'Time': datetime.now().strftime('%H:%M:%S')}]).to_csv(log_file,
                            st.success(f"✅ MARKED PRESENT: {student['Name']}")
                            recognized = True
                            break
                if not recognized:
                    st.error("❌ Face not recognized. Are you registered? (If yes, try better lighting).")

# --- TAB 3: REPORT ---
with tab3:
    st.header("Dropout Analysis Report")
    if st.button("Generate Today's Report"):
        if not os.path.exists(DB_PATH):
            st.warning("No students in the database.")
        else:
            m = pd.read_csv(DB_PATH)
            l = f"attendance_{datetime.now().strftime('%Y-%m-%d')}.csv"
            if not os.path.exists(l):
                st.warning("No attendance marked today yet.")
            else:
                att = pd.read_csv(l).drop_duplicates(subset=['ID'])
                st.markdown("### 📊 Daily Dropout Rates")
                for g in ["Boy", "Girl"]:
                    tot = len(m[m['Gender'] == g])
                    pre = len(att[att['Gender'] == g])
                    rate = ((tot - pre) / tot * 100) if tot > 0 else 0
                    st.markdown(f"- **{g}s:** {pre}/{tot} present (**{rate:.1f}% Absent/Dropout**)")
```

requirements.txt



3. GitHub

Link: https://github.com/Anshikabhadauria113/attendance-tracker-ai

CONCLUSION

The Smart Student Portal successfully demonstrates that computer vision can be utilized to streamline administrative tasks in educational settings. By prioritizing lightweight algorithms (LBPH) over heavy deep-learning models, the project achieved a 100% stable cloud deployment.